

Presented By:
JB Parrett
Principal Software Developer

POWERED

Palm OS® Cobalt Details: Schema Databases and IPC

The contents of and the technologies described in this presentation are provided to you and described "AS IS," without any warranty or representation of any kind. PalmSource hereby disclaims each and every other representation or warranty with respect thereto, including the implied warranties of merchantability, fitness for a particular purpose, accuracy and non-infringement. No representation or warranties are made that any product based in whole or in part on any specifications or information referenced herein will be free from defects or safe for use for its intended purposes.

PalmSource further reserves the right to modify in any respect any technology described or content provided herein without prior written notice.

Palm OS is a registered trademark and Palm Powered is a trademark of Palm Trademark Holding Company, LLC. Other brands may be trademarks of their respective owners. All other trademarks and tradenames used herein are the property of their respective owners.

Copyright (c) 2004 PalmSource, Inc.



Overview

Schema Databases

IPC

Q&A



Databases in Palm OS® Cobalt

Classic

- Same as in Palm OS® Garnet (“Palm OS® 5”) and earlier

Extended

- Only available for native applications
- Data format uses native endianness
- Uniquely identified by name and creator code
- Records can exceed 64K in size
- Still limited to 64K records per database

Schema



Schema Databases

Data abstraction via schemas

- Easier sharing of data across applications
- Easier to synchronize with desktop
- Easily extended with additional columns

System services

- Data Manager provides optimized, internationalized sorting

Binding

- Column data to variables
- Row change automatically updates variables

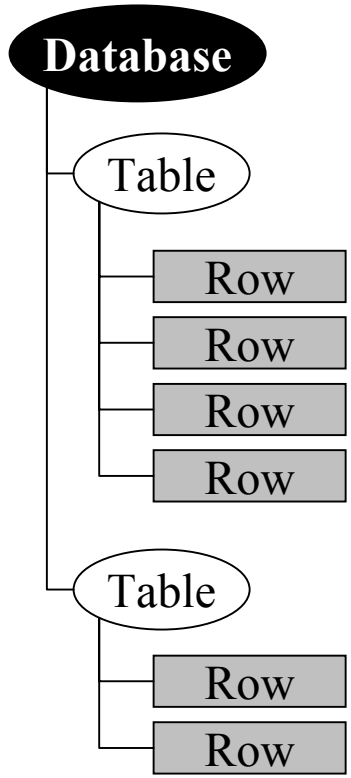
Categories

- Up to 255
- Multiple categories per row is allowed

Security

Nomenclature

- Data stored in *Tables*
- Tables are collections of *Columns* and *Rows*
- Rows exist in only one table
- Columns have well known data types
- Each row may have a value for each column in its table
- Databases may have multiple tables





Tables

Name	Age	Phone #
Joe	34	555-4857
Mary		(512) 555-8732
John	56	555.0254
Bob	19	800-555-4BOB
Ann	24	



Sort Indices

Multiple sort indices allowed for each table

- Performance considerations
- Can be added, removed, or disabled after DB creation

Supports two-key indices

- Each key may consists of multiple columns
- All key columns must have the same type

Automatically updated when database changes

Sort indices are stored with database

Default index is unordered



Cursors

Iterate over subset of rows in a table

Created using existing sort index

Creation may also include selection query

Row contents are not protected against changes

- Can refresh the cursor if rows are modified

First row of a cursor is 1 (not 0!)



Cursor API

```
DbCursorMoveFirst(myCursor);

while(!DbCursorIsEOF(myCursor)){
  DbGetColumnValues(dbRef, myCursor, ...);
  DbCursorMoveNext(myCursor);
}

DbCursorSetAbsolutePosition(myCursor, pos);
```



SQL Support

Not currently compliant with SQL standard

Describe/manipulate sort indices (ORDER BY)

Filter rows in a cursor (WHERE)

Examples:

```
SELECT * FROM table WHERE age = 24

table ORDER BY name, age

table WHERE age = 24 ORDER BY last_name

Names ORDER BY FName, LName DESC, (102, 103) CASELESS
```



Extended Category Support

Up to 255 categories per database

Rows may belong to multiple categories

***Unfiled* no longer considered a category**

- Indicates lack of membership in any category

Stored as part of database

- No AppInfo block
- Can store non-category data from classic AppInfo in separate table in database



Category Example

```
sqlStr = "Expenses WHERE Amount < 75";
flags = dbCursorOnlySecret | dbCursorSortByCategory;
catIDs = { 3, 6, 119 };
matchMode = DbMatchAll;

err = DbCursorOpenWIthCategory(dbRef, sqlStr, flags, 3,
   catIDs, matchMode, &myCursor);

DbCursorMoveFirst(myCursor);

while(!DbCursorIsEOF(myCursor)){
   DbGetColumnValues(dbRef, myCursor, ...);
   DbCursorMoveNext(myCursor);
}
```



Concurrency

- All operations serialized per database by the server
- Synchronization still required if multiple threads using the same open database reference
- A database may be opened for reading multiple times
- Only one open reference can have write access
- Applications may choose not to share



Secure Databases

- All access gated by the security services
- Stored in protected area only accessible by Data Manager
- Reads & writes copy all data
- Large performance overhead
- Not encrypted on device
- Encrypted when backed up if a password is set



Performance Tips

Read *Memory, Databases, and Files*

Don't open & close databases or cursors frequently

Avoid copying column values

Use `DbGetColumnValues()`

- Not `DbCopyColumnValues()`

Use `DbGetColumnValues()`

- Not `DbGetColumnValue()` multiple times

Why?



Inter-Process Communication (IPC)

Palm OS® Cobalt is designed with a protected memory system

Benefits:

- Security (can prevent data access from another process)
- Stability (a crash in one process won't kill the system)
- Maintainability (reduces dependencies)
- Convenience (OS can track & release resources easily)

Impact:

- Optimization opportunity!
- You won't feel the impact in most cases
- We help out where we can



Palm OS Cobalt Processes

Data Mgr	System	I/O	Application	Background	Sublaunch x3	Licensee
Data Mgr threads	Window Mgr	IOS	UI application	Comms thread(s)	Notification Agent	License UI Component
	App Mgr	VFS	PACE	Exchange Sublaunch	Find Agent	Licensee Srv
	Event Mgr	STREAMS framework	OS Shared libraries	Slips	PACE	
	AM/AZM	STREAMS components		Sound callbacks		
	Status Bar	HW device drivers		PACE		
	Feature Srv					
	Exchange Srv					
	Notification Srv					
Local Heap	**Local Heap**	**Local Heap**	**Local Heap**	**Local Heap**	**Local Heap**	**Local Heap**

System Shared Memory

Secure DBs | **Frame Buffer** | **Audio Mixer** | **Legacy Heap (on request)**

Storage Heap

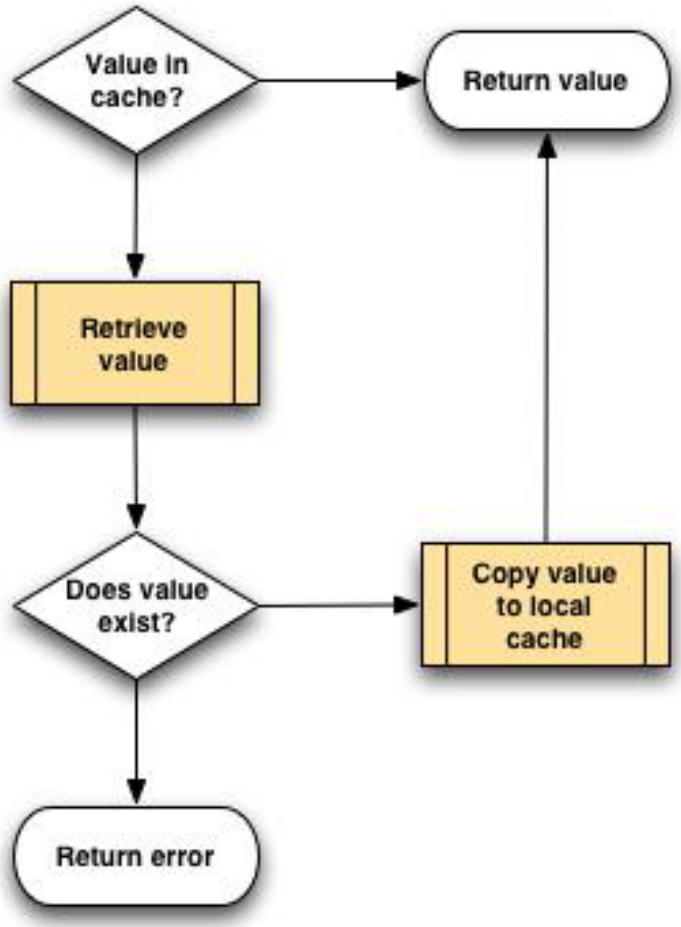
ROM

Examples of IPC - `FtrGet()`

`FtrGet()` does IPC to retrieve a value the first time

The system caches the retrieved value

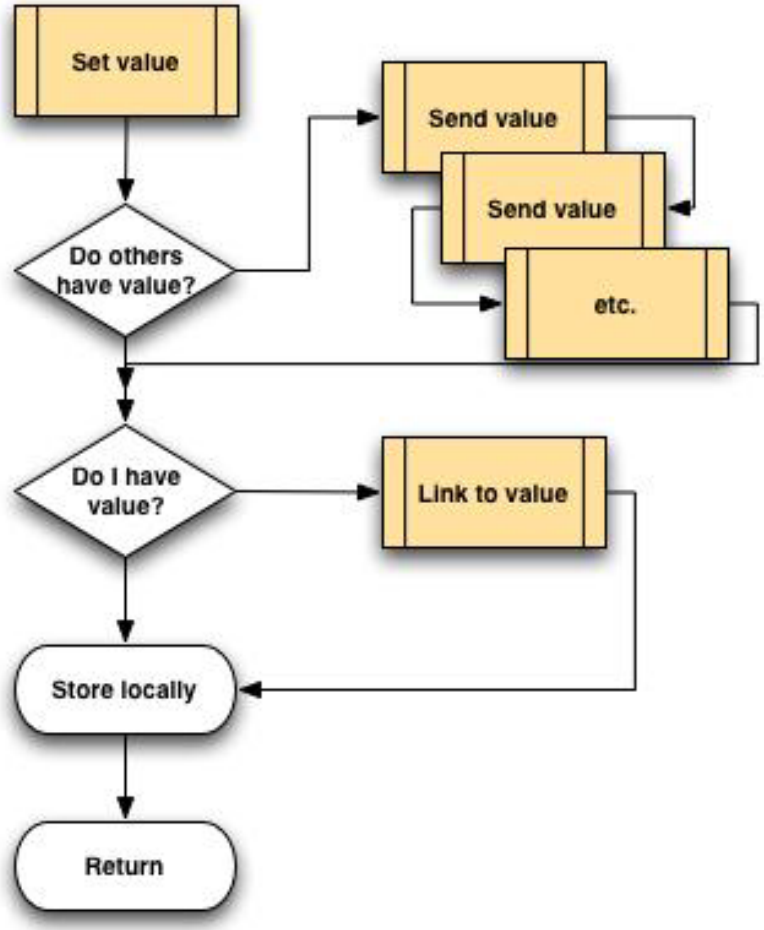
Subsequent calls for the same feature are fast



Examples of IPC - `FtrSet()`

Since multiple processes may have local copies of a given feature, `FtrSet()` is slow

Must broadcast changed value to all processes that have a copy



Examples of IPC - `FtrPtrNew()`

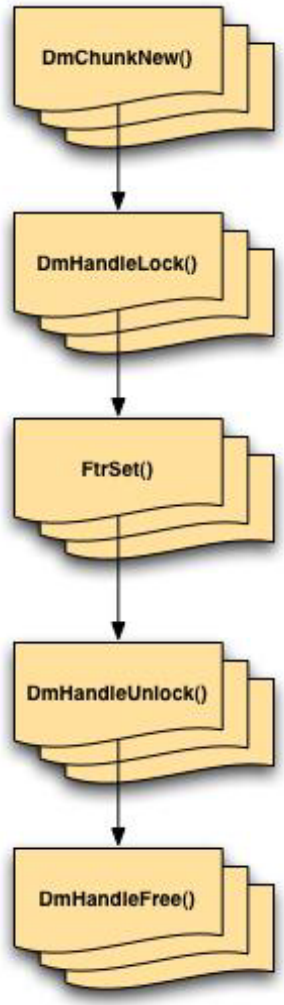
`FtrPtrNew()` requires the Data Manager

- Separate process
- Also calls FtrSet()

As a result, `FtrPtrNew()` and other Feature memory calls are slower than before

Better to use the dynamic heap

- It is a real heap now!
- Faster
- Not limited to 64K chunks





IPC Optimization - Graphics Context

The rendering engine runs in a different process

- But some apps need to make lots of drawing calls

To avoid IPC overhead:

- System buffers drawing calls
- Sends them in batches at the end of the event loop

Note: this means that drawing might not be done when GcXXX() functions return

- `GcCommit()` and `GcFlush()` are there if you need them



Q & A